Sardar Patel Institute of Technology
Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai-400058-India
(Empowered Autonomous Institute Affiliated to University of Mumbai)
Academic year 2026-2027

Name: Chinmay Tawade | UID: 2024200132 | Batch: B4

Experiment 5

Steganography and Cryptography

Hiding Secrets in Plain Sight — Photos, Videos and Python

AIM

To hide secret data inside harmless-looking carrier files (images and video) using both a password-protected tool (OpenStego) and custom Python least-significant-bit (LSB) code, to recover the data, and to measure how much the carrier changes.

OBJECTIVES

1. Embed and extract a text message with OpenStego using AES-128 password protection, and observe the wrong-password behaviour.
2. Hide a text message in a PNG cover image using Python LSB embedding and verify exact recovery.
3. Hide an image inside another image using 1, 2, 4 and 7 bit-planes, and hide an image inside a video frame with a lossless codec (FFV1) and a lossy codec (MP4V).
4. Quantify the distortion with SHA-256 hashes, changed-pixel count, exact pixel-match percentage, mean squared error (MSE) and a difference map, against the targets of accuracy above 95% and MSE below 1%.

REQUIREMENTS

Item	Details
Operating system	Ubuntu (lab machine); Google Colab can be used for the Python parts
Tools	OpenStego (GUI) for password-protected embedding and extraction
Python packages	pillow, numpy, opencv-python (python -m pip install pillow numpy opencv-python)
Input files	secret.txt, cover.jpg (OpenStego); cover_image.png, secret_image.png, cover_video.mp4 (Python)
Code files	stego_lab.py (core functions), run_lab.py, run_video.py, run_forensics.py

THEORY

Steganography versus cryptography

Cryptography makes a message unreadable; steganography hides the fact that a message exists. Used together, the secret is first encrypted and then hidden, so an observer must both notice the hidden data and break the encryption.

Aspect	Cryptography	Steganography
Goal	Confidentiality of content	Concealment of existence
Output looks like	Random-looking ciphertext	An ordinary image or video
If detected	Attacker knows a secret exists	Nothing suspicious is seen

Aspect	Cryptography	Steganography
Example here	AES-128 password protection in OpenStego	LSB embedding in PNG pixels and video frames

LSB and bit-plane embedding

Each pixel has three 8-bit channels (R, G, B). The least significant bit changes the channel value by at most 1, which the eye cannot see. Embedding one bit uses clearing and inserting:

```
stego = (cover & 0xFE) | secret_bit        # 1 bit per channel value
```

Using k bit-planes keeps the upper (8 − k) bits of the cover and stores k bits of the secret in the lower bits, so a channel changes by at most 2^k − 1. More planes carry more data but cause visible distortion.

Bits (k)	Max change per channel (2^k − 1)	Capacity, 1200×1500 cover (bytes)	Capacity, 935×623 cover (bytes)
1	1	675,000	218,439
2	3	1,350,000	436,878
4	15	2,700,000	873,757
7	127	4,725,000	1,529,076

Theoretical capacity = width × height × 3 channels × k / 8 bytes, before any header or length field.

Lossless versus lossy carriers

LSB data survives only if the carrier is stored losslessly. PNG and the FFV1 video codec preserve every pixel value exactly, whereas JPEG and MP4V compression re-quantise pixels and can destroy the hidden bits. This is why OpenStego writes a PNG output and why the video experiment compares FFV1 with MP4V.

Encrypting before embedding

The custom Python LSB functions do not encrypt anything; they only hide bits. Real protection comes from encrypting first, preferably with authenticated encryption such as AES-GCM (confidentiality plus tamper detection), and then embedding the ciphertext. The carrier must have enough capacity and the receiver needs the payload length. OpenStego's AES-128 password option is a separate, built-in implementation of this idea.

Forensic measures

- **SHA-256:** file fingerprint; any changed bit changes the hash completely.
- **Exact pixel match:** percentage of pixels whose R, G and B values are all unchanged.
- **MSE:** mean of squared differences on the 0–255 scale; normalised MSE (%) = MSE / 255² × 100.
- **Difference map:** image of per-pixel differences, amplified so single-bit changes become visible.

PROCEDURE

A. OpenStego (password-protected)

1. Open OpenStego and choose Hide data.
2. Select the message file (secret.txt), the cover file (cover.jpg) and the output stego file (stego.png).
3. Keep the encryption algorithm AES128, enter and confirm a password, and click Hide data.
4. Confirm the output with ls -lh and file.

5. Choose Extract data, select stego.png, set the output folder (recovered) and enter the password; verify the recovered file with cat.
6. Repeat extraction with a wrong password and note the error.

B. Python LSB experiments

1. Run run_lab.py: hide "Hello, this is a hidden message!" in cover_image.png, decode it, then hide secret_image.png using 1, 2, 4 and 7 bits and compare each stego image with the cover.
2. Run run_video.py: embed the secret image in the first video frame with FFV1 (lossless) and MP4V (lossy), then extract with the same bit count.
3. Run run_forensics.py: compute SHA-256 hashes, changed pixels, exact match, MSE and the difference map.

OBSERVATIONS AND RESULTS

1. OpenStego: hiding secret.txt in cover.jpg

The cover photograph was used unchanged as the carrier (Figure 1). OpenStego was configured with the message file, cover file and output file, AES128 encryption and a password.



Figure 1. *Cover image cover.jpg (935 × 623). Photo credit as shown on the image: Red Bull Content Pool.*

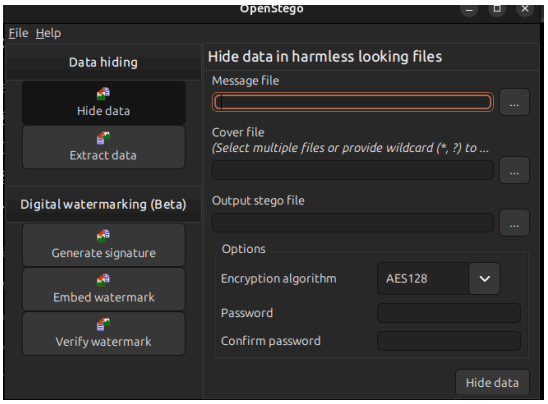


Figure 2. *OpenStego "Hide data" interface.*

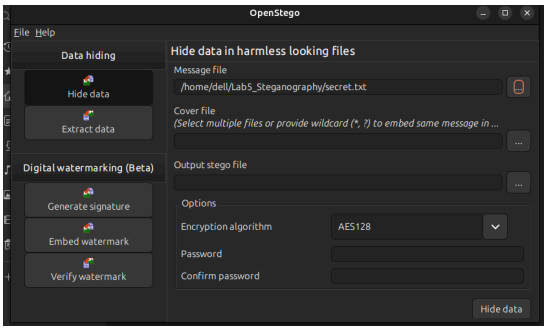


Figure 3. *Message file secret.txt selected.*

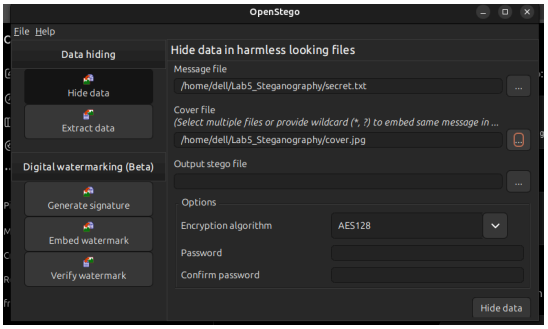


Figure 4. *Cover file cover.jpg selected.*

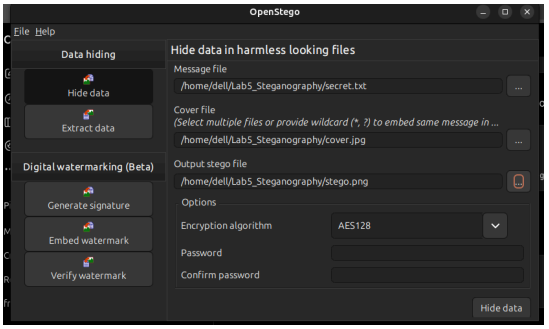


Figure 5. *Output stego file stego.png set.*

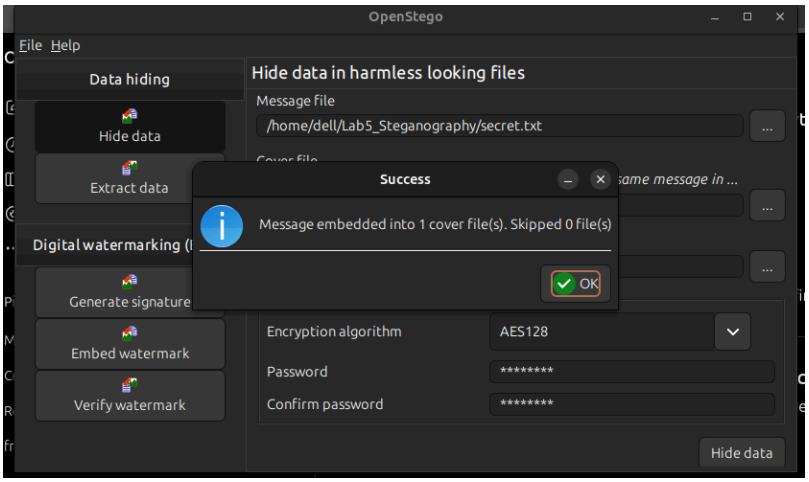


Figure 6. *Success: message embedded into 1 cover file, 0 skipped (AES128, password entered).*

```
dell@dell-OptiPlex-Tower-7020:~/Lab5_Steganography$ cd ~/Lab5_Steganography
ls -lh cover.jpg stego.png
-rw-rw-r-- 1 dell dell 573K Oct  2 09:53 cover.jpg
-rw-rw-r-- 1 dell dell 799K Oct  2 10:11 stego.png
dell@dell-OptiPlex-Tower-7020:~/Lab5_Steganography$ file stego.png
stego.png: PNG image data, 935 x 623, 8-bit/color RGB, non-interlaced
```

Figure 7. *cover.jpg (573 KB) and stego.png (799 KB, PNG 935 × 623, 8-bit RGB).*

Extraction used the correct password and recovered the original file; a wrong password produced an error.

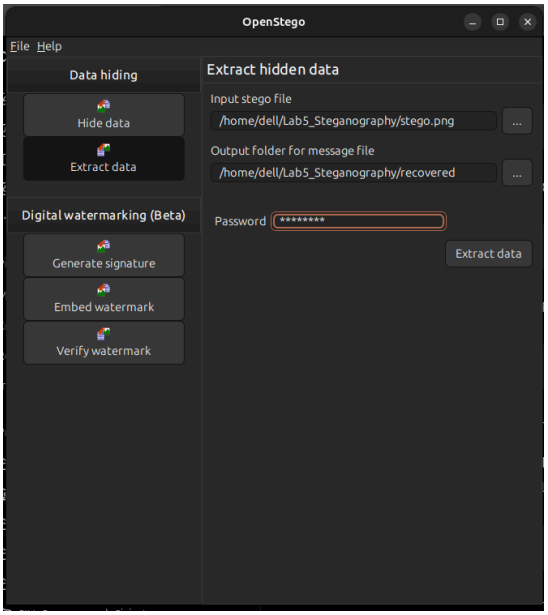


Figure 8. *Extract data: stego.png, output folder recovered, password entered.*

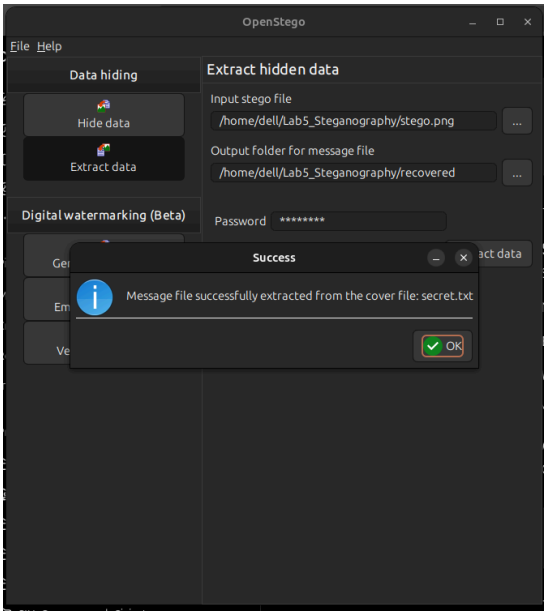


Figure 9. *Success: secret.txt extracted from the cover file.*

```
dell@dell-OptiPlex-Tower-7020:~/Lab5_Steganography$ cd ~/Lab5_Steganography
ls -lh recovered/
total 4.0K
-rw-rw-r-- 1 dell dell 64 Oct  2 10:15 secret.txt
dell@dell-OptiPlex-Tower-7020:~/Lab5_Steganography$ cat recovered/*
Chinmay Tawade
Lab 5 - Steganography
This is my hidden message.
```

Figure 10. *Recovered secret.txt (64 bytes) shows the original message.*

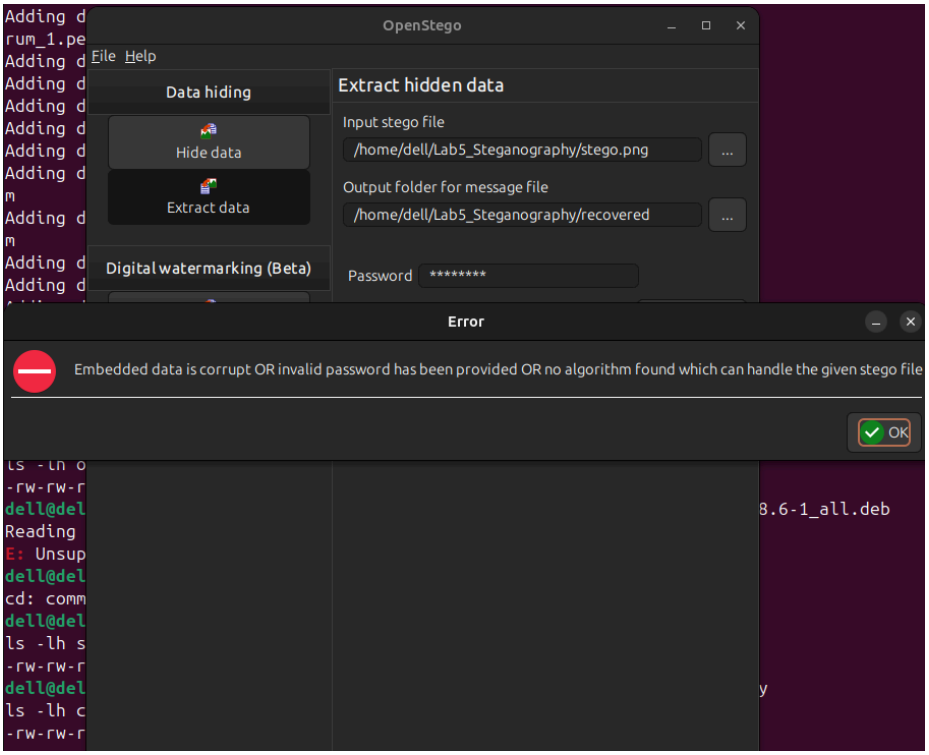


Figure 11. *Wrong password: extraction fails with a corrupt/invalid-password error.*

Parameter	Observation
Message file	secret.txt, 64 bytes (three lines of text)
Cover file	cover.jpg, 573 KB, 935 × 623
Stego file	stego.png, 799 KB, PNG, 935 × 623, 8-bit RGB
Encryption / password	AES128, password protected
Extraction, correct password	Success; recovered secret.txt, 64 bytes, content identical to the original message
Extraction, wrong password	Error: embedded data corrupt OR invalid password OR no algorithm found

The stego file is larger than the cover because the JPEG was decoded and re-saved as a lossless PNG; the size increase is a format change, not extra data. The difference map below shows that only a very small fraction of pixels differ from the cover.

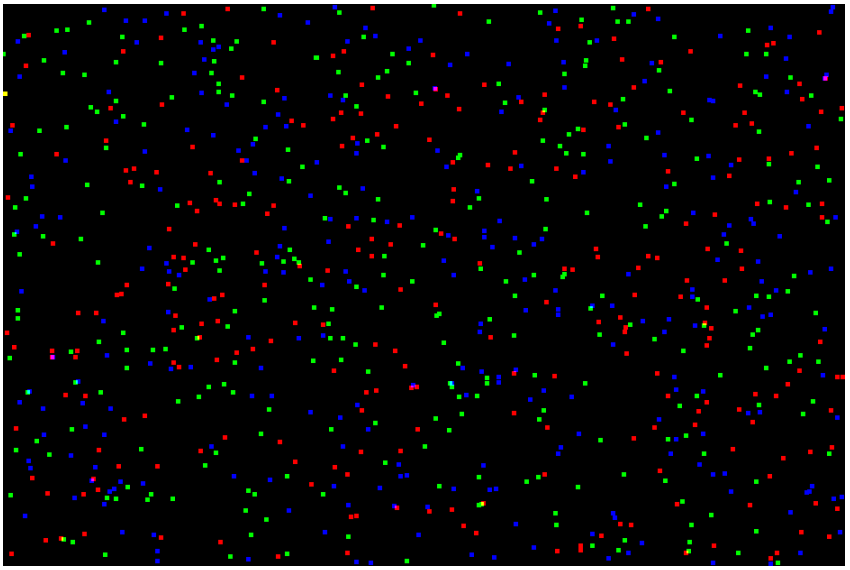


Figure 12. *Amplified difference map for the 935 × 623 cover: about 837 pixels (0.14%) are non-zero (dots enlarged for visibility).*

2. Python: hiding a text message in cover_image.png

The message "Hello, this is a hidden message!" (32 bytes) was hidden in cover_image.png and saved as encoded_image.png. The two images look identical (Figures 13 and 14). The measurements below were computed from the supplied cover_image.png and encoded_image.png.



Figure 13. *cover_image.png (original, 1200 × 1500).*



Figure 14. *encoded_image.png (message hidden).*

Measure	Value
Image size	1200 × 1500 pixels (1,800,000 pixels)
File size, cover_image.png	1,450,113 bytes
File size, encoded_image.png	1,450,224 bytes (111 bytes larger)
Changed pixels	89 pixels (149 channel values), all in row 0, columns 0–95
Maximum difference in any channel	1 (LSB only)
Exact pixel match	99.9951 %
Raw MSE	2.76×10^{-5}
Normalised MSE	4.24×10^{-8} %
PSNR	93.7 dB
SHA-256, cover_image.png	eaff4df15dc6d9ac849922e339ed82ddec6ab665a9672268cad500876bba8382
SHA-256, encoded_image.png	68fde7000b0d4c116a02ab3c1e53abdda5870acf88316f96dbdc59b9b2acd046

The hashes differ even though the pictures look the same, so hash comparison detects the change while the eye cannot. The changes are confined to the first 96 pixels of the top row, which equals 288 bits: the 32-byte message plus the 4-byte length header is 36 bytes, or 288 bits, one bit per colour value across 96 pixels. Both targets are met: exact match 99.995% (above 95%) and normalised MSE far below 1%.



Figure 15. *Difference map of the top row (columns 0–95, magnified): changed pixels are confined to the first 96 pixels.*

3. Image-in-image hiding with 1, 2, 4 and 7 bits

The secret image is resized to the cover size and its top k bits are stored in the cover's lowest k bit-planes. Recovery is quantised, so exact pixel matching is not expected; the secret should be compared after resizing it to the cover dimensions.

Bits (k)	Max change / channel	Changed pixels	Raw MSE (cover vs stego)	Normalised MSE (%)
1	1	—	—	—
2	3	—	—	—
4	15	1,798,783 (99.93%)	55.19	0.0849
7	127	—	—	—

The 4-bit row is measured from the supplied stego_image_bits4.png (cover_image.png vs stego image; maximum observed difference 15, PSNR 30.7 dB). Highlighted cells are for the 1-, 2- and 7-bit runs and are filled with the values printed by run_lab.py. The maximum-change column is theoretical (2^k − 1).

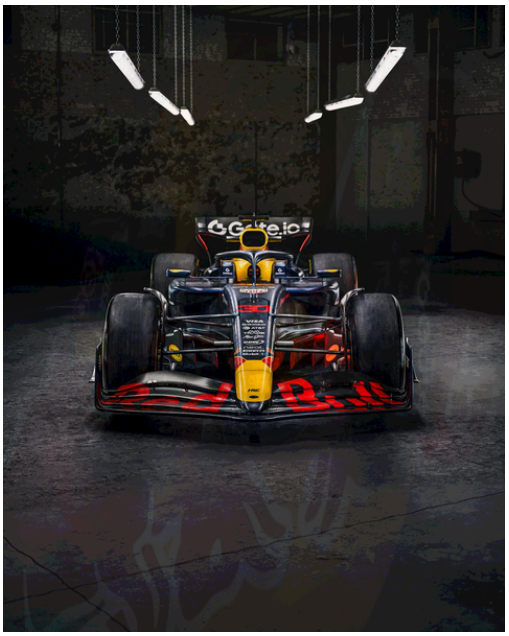


Figure 16. *stego_image_bits4.png: cover with the secret image hidden using 4 bits per colour value.*

At 4 bits almost every pixel (99.93%) is altered, by up to 15 levels, so the exact pixel match with the cover is only 0.07%. The raw MSE is 55.19, which is 0.085% when normalised and well below the 1% target, and the PSNR is 30.7 dB. The cover still looks like the original car, but a faint ghost outline of the secret picture can be seen on the floor and walls (Figure 16). This is the sandwich trade-off: the more layers given to the secret, the more the cover is damaged. Note that the 95% exact-match target applies to the text experiment; for image-in-image hiding exact pixel match is not expected.

4. Image-in-video hiding: lossless (FFV1) versus lossy (MP4V)

The secret image was embedded in the first frame of cover_video.mp4 with 4 bits and extracted from two videos: the lossless FFV1 output (encoded_video.avi) and the lossy MP4V output. Figures 17 and 18 show the two extracted images.

Video link : link



Figure 17. *extracted_image.png: recovered from the lossless FFV1 video.*



Figure 18. *lossy_extracted.png: recovered from the lossy MP4V video.*

Comparison (lossless vs lossy extraction, 640 × 640)	Value
Exact pixel match	5.80 %
Raw MSE	791.4
Normalised MSE	1.217 %
Largest difference in any channel	240 (out of 255)

The original secret_image.png was not part of the supplied files, so the two extractions are compared with each other. Both are quantised copies of the secret because only its top 4 bits were stored.

The lossless extraction is a smooth, if low-detail, copy of the secret. The lossy extraction is still recognisable (the car and the logo can be made out) but it is visibly degraded: colours are shifted and over-saturated, shadows are crushed, and blocky compression noise appears. Only 5.8% of its pixels match the lossless result. MP4V compression re-quantises every frame, which alters the low bits that carry the secret, and the extraction stretches the low 4 bits back to full brightness, so even small compression errors become large visible errors. A picture tolerates this kind of damage; an encrypted file or a text message with a length header would be corrupted beyond recovery. Lossless carriers such as FFV1 are therefore required for reliable hiding.

ANALYSIS

- Text hiding is almost undetectable: only 149 of 5,400,000 channel values changed, each by 1. Exact recovery is possible because the payload is only a few bytes.
- OpenStego combines hiding with encryption: the correct password recovered the 64-byte file, and a wrong password failed. The error message does not say whether the password or the data was at fault, which reveals nothing useful to an attacker.
- The embedded payload is detectable only by comparing against the original cover (hash, difference map). Without the original, a single hash or glance shows nothing unusual.
- Capacity and invisibility trade off: more bit-planes carry more data but change each channel by up to 2^k − 1.
- Format matters: PNG and FFV1 preserve hidden bits. With MP4V the secret image was only partly recoverable, with 5.8% pixel match to the lossless extraction.

- At 4 bits the cover changed by up to 15 levels per channel (PSNR 30.7 dB) and a faint ghost of the secret became visible, so 4 bits already trades noticeable cover quality for capacity.
- Plain LSB hiding can be found by statistical steganalysis, so valuable secrets should be encrypted before they are embedded.

CRYPTOGRAPHY AND STEGANOGRAPHY TOGETHER

Step	Operation	Detail
1	Secret	The file or image to protect
2	Encrypt	Authenticated encryption (for example AES-GCM) with a secret key; OpenStego does this with its AES128 option
3	Embed	Hide the ciphertext in a carrier image or video frame; check capacity and store the payload length
4	Extract	Receiver recovers the ciphertext using the same bit count (and frame index for video)
5	Decrypt and verify	Decrypt with the key and verify the recovered file, for example by comparing hashes

In this experiment, hide_image() and hide_in_video() only embed bits and do not encrypt; the encryption step was demonstrated through OpenStego's AES-128 password protection.

CONCLUSION

Secret data was hidden and recovered with both OpenStego and custom LSB code. OpenStego recovered the 64-byte secret.txt with the correct password and rejected a wrong one. In the Python text experiment only 89 of 1,800,000 pixels changed, by at most 1 per channel, giving 99.995% exact pixel match and a normalised MSE of about 4×10^{-8} %, well within the targets of above 95% and below 1%. Hiding a whole image with 4 bits changed nearly every pixel (normalised MSE 0.085%, PSNR 30.7 dB) and left a faint ghost of the secret in the cover. The lossy MP4V video degraded the extracted secret heavily (1.2% normalised MSE against the lossless extraction), confirming that lossless carriers are essential. Steganography hides that a message exists, so for real confidentiality it should be combined with encryption.

APPENDIX — CODE

All code below is listed in the order of the lab: stego_lab.py (text, image and video functions, then the forensic helpers) followed by the three run scripts.

stego_lab.py — part 1: hide and reveal text

```
from PIL import Image
import numpy as np
import cv2

def encode_text(cover_path, message, out_path):
    """Hide `message` in the LSBs of the cover image. Save as PNG (lossless!)."""
    pixels = np.array(Image.open(cover_path).convert("RGB"))
    flat = pixels.flatten()                        # every colour value in one long row
    data = message.encode("utf-8")
    header = len(data).to_bytes(4, "big")          # first 4 bytes = message length
    bits = np.unpackbits(np.frombuffer(header + data, dtype=np.uint8))
    if len(bits) > flat.size:
        raise ValueError("Message is too long for this image. Use a bigger image!")
    flat[:len(bits)] = (flat[:len(bits)] & 0xFE) | bits   # overwrite the last bit
    Image.fromarray(flat.reshape(pixels.shape)).save(out_path)
```

```
    print(f"Hidden {len(data)} bytes inside {out_path}")

def decode_text(stego_path):
    """Read the LSBs back and rebuild the message."""
    flat = np.array(Image.open(stego_path).convert("RGB")).flatten()
    length = int.from_bytes(np.packbits(flat[:32] & 1).tobytes(), "big")
    bits = flat[32:32 + length * 8] & 1
    return np.packbits(bits).tobytes().decode("utf-8")
```

stego_lab.py — part 2: hide an image inside an image

```
def hide_image(cover_path, secret_path, out_path, bits=4):
    """Put the top `bits` bits of the secret into the low `bits` bits of the cover."""
    cover = Image.open(cover_path).convert("RGB")
    secret = Image.open(secret_path).convert("RGB").resize(cover.size)
    c, s = np.array(cover), np.array(secret)
    keep_mask = (0xFF << bits) & 0xFF             # bits=4 -> 11110000
    stego = (c & keep_mask) | (s >> (8 - bits))
    Image.fromarray(stego.astype(np.uint8)).save(out_path)
    print(f"Secret image hidden in {out_path} (using {bits} bits per colour)")

def reveal_image(stego_path, out_path, bits=4):
    """Pull the low bits out and stretch them back to full brightness."""
    st = np.array(Image.open(stego_path).convert("RGB"))
    secret = (st & ((1 << bits) - 1)) << (8 - bits)
    Image.fromarray(secret.astype(np.uint8)).save(out_path)
    print(f"Secret image recovered -> {out_path}")
```

stego_lab.py — part 3: hide an image inside a video

```
def hide_in_video(video_in, secret_path, video_out, bits=4, codec="FFV1"):
    cap = cv2.VideoCapture(video_in)
    w, h = int(cap.get(3)), int(cap.get(4))
    fps = cap.get(cv2.CAP_PROP_FPS)
    secret = cv2.resize(cv2.imread(secret_path), (w, h))
    out = cv2.VideoWriter(video_out, cv2.VideoWriter_fourcc(*codec), fps, (w, h))
    keep_mask = (0xFF << bits) & 0xFF
    first = True
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        if first:                                 # hide only in frame #1
            frame = (frame & keep_mask) | (secret >> (8 - bits))
            first = False
        out.write(frame)
    cap.release()
    out.release()
    print(f"Secret hidden in first frame of {video_out}")

def reveal_from_video(video_path, out_path, bits=4):
    cap = cv2.VideoCapture(video_path)
    ok, frame = cap.read()
    cap.release()
    if not ok:
        raise RuntimeError("Could not read the video")
    secret = (frame & ((1 << bits) - 1)) << (8 - bits)
    cv2.imwrite(out_path, secret.astype(np.uint8))
    print(f"Secret recovered -> {out_path}")
```

stego_lab.py — forensic helpers

```
import hashlib

def sha256_file(path):
    """Calculate the SHA-256 hash of a file."""
    digest = hashlib.sha256()
    with open(path, "rb") as f:
        for chunk in iter(lambda: f.read(1024 * 1024), b""):
            digest.update(chunk)
    return digest.hexdigest()

def compare_images(original_path, modified_path, prefix="diff"):
    """Calculate pixel MSE and save a visual difference map."""
    a = np.array(Image.open(original_path).convert("RGB"), dtype=np.int16)
```

```
    b = np.array(Image.open(modified_path).convert("RGB"), dtype=np.int16)
    if a.shape != b.shape:
        raise ValueError("Images must have identical dimensions.")
    difference = np.abs(a - b)
    diff_map = np.clip(difference.max(axis=2) * 8, 0, 255).astype(np.uint8)
    Image.fromarray(diff_map).save(f"{prefix}_map.png")
    mse = float(np.mean((a.astype(float) - b.astype(float)) ** 2))
    normalized_mse_percent = mse / (255.0 ** 2) * 100.0
    exact_pixel_match = float(np.mean(np.all(a == b, axis=2)) * 100.0)
    changed_pixels = int(np.count_nonzero(np.any(a != b, axis=2)))
    print("Changed pixels:", changed_pixels)
    print(f"Exact pixel match: {exact_pixel_match:.4f}%")
    print(f"Raw MSE: {mse:.6f}")
    print(f"Normalized MSE: {normalized_mse_percent:.6f}%")
    print(f"Difference map: {prefix}_map.png")
    return mse, normalized_mse_percent, exact_pixel_match
```

run_lab.py

```
from stego_lab import *

encode_text("cover_image.png", "Hello, this is a hidden message!", "encoded_image.png")
print("Hidden message:", decode_text("encoded_image.png"))

for bits in (1, 2, 4, 7):
    stego_file = f"stego_image_bits{bits}.png"
    recovered_file = f"recovered_secret_{bits}.png"
    hide_image("cover_image.png", "secret_image.png", stego_file, bits=bits)
    reveal_image(stego_file, recovered_file, bits=bits)
    compare_images("cover_image.png", stego_file, prefix=f"cover_stego_{bits}")
```

run_video.py

```
from stego_lab import *

# Lossless (recommended)
hide_in_video("cover_video.mp4", "secret_image.png", "encoded_video.avi", bits=4, codec="FFV1")
reveal_from_video("encoded_video.avi", "extracted_image.png", bits=4)

# Lossy comparison
hide_in_video("cover_video.mp4", "secret_image.png", "lossy_video.avi", bits=4, codec="mp4v")
reveal_from_video("lossy_video.avi", "lossy_extracted.png", bits=4)
```

run_forensics.py

```
from stego_lab import *

for filename in ("cover_image.png", "encoded_image.png"):
    print(filename, "SHA-256:", sha256_file(filename))

compare_images("cover_image.png", "encoded_image.png", prefix="text_stego")
```